

AER1415: Computational Optimization

Assignment 1

University of Toronto Institute for Aerospace Studies

Zouhair Adam Hamaimou
Student Number: 1004981986

Due: February 14, 2025

Contents

1	Introduction	2
2	PSO Algorithm Implementation and Testing	2
2.1	Algorithm Description	2
2.2	Parameter Tuning Studies	3
2.3	Results for Case Studies	6
2.3.1	P1: Minimization of the Rosenbrock Function	6
2.3.2	P2: Constrained Optimization Problem (Algebraic in 2 Variables) . .	8
2.4	Results for P3: Constrained Bump Test Function	9
2.5	P4: Brachistochrone Problem	9
2.5.1	Problem Formulation	10
2.5.2	Results of PSO for the Brachistochrone Problem	10
2.6	P5: Estimation of Damping Factor and Stiffness	13
2.6.1	Problem Formulation	13
2.6.2	Optimization Results	14
3	Regression Problem	15
3.1	Matrix Formulation	15
3.2	Rewriting the Objective Function	16
3.3	Computing the Gradient	16
3.4	Solution for $\mathbf{w}$	17
4	Constraint Relaxation Optimization Problem	17
4.1	Minimizing the Total Relaxation	17
4.2	Minimizing the Number of Relaxed Constraints	17
4.3	Minimizing the Cost of Relaxation	18
5	Bonus: Particle Swarm Optimization on P3 ($n = 50$)	18

1 Introduction

This report presents solutions to Assignment 1 for the AER1415 course, focusing on the implementation and testing of the Particle Swarm Optimization (PSO) algorithm for constrained optimization problems. The following sections describe the PSO algorithm, parameter tuning studies using the bump test function (P3), and the application of the optimized parameters to other case studies. Additionally, solutions to the regression problem and the constraint relaxation problem are included.

2 PSO Algorithm Implementation and Testing

2.1 Algorithm Description

Particle Swarm Optimization (PSO) is a stochastic optimization algorithm inspired by the collective behavior of swarms. The algorithm maintains a population of particles, each representing a potential solution in the search space. The particles adjust their positions based on their own experiences and the experiences of their neighbors.

For the following general constrained optimization problem:

$$\begin{aligned} \text{Minimize: } & f(\mathbf{x}) \\ \text{Subject to: } & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m, \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, q, \\ & \mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n, \end{aligned}$$

The main steps of the PSO algorithm are as follows:

1. Initialize a swarm of particles with random positions $\mathbf{x}_i(t=0)$ and velocities $\mathbf{v}_i(t=0)$ within the defined bounds.
2. Evaluate the objective function value $f(\mathbf{x}_i)$ for each particle.
3. Update each particle's personal best position $\mathbf{p}_{i,\text{best}}$ and the global best position $\mathbf{g}_{\text{best}}$.
4. Update the velocity and position of each particle using:

$$\begin{aligned} \mathbf{v}_i(t+1) &= w\mathbf{v}_i(t) + c_1\mathbf{r}_1 \odot (\mathbf{p}_{i,\text{best}} - \mathbf{x}_i(t)) + c_2\mathbf{r}_2 \odot (\mathbf{g}_{\text{best}} - \mathbf{x}_i(t)), \\ \mathbf{x}_i(t+1) &= \mathbf{x}_i(t) + \mathbf{v}_i(t+1), \end{aligned}$$

where w is the inertia weight, c_1 and c_2 are acceleration coefficients, and $\mathbf{r}_1$ and $\mathbf{r}_2$ are random vectors whose elements are uniformly distributed in the interval $[0, 1]$.

5. Repeat steps 2–4 until the maximum number of iterations is reached or the improvement in the objective value is less than the specified tolerance for a defined number of consecutive iterations.

To handle constraints, the quadratic penalty function approach is employed:

$$\phi(\mathbf{x}) = \sum_{i=1}^m \max(0, g_i(\mathbf{x}))^2 + \sum_{i=1}^q h_i(\mathbf{x})^2,$$

The penalty is added to the objective function value using a static or adaptive penalty coefficient to discourage infeasible solutions. The static penalty method applies a fixed penalty parameter λ throughout the optimization process. The penalized objective function is given by:

$$F(\mathbf{x}) = f(\mathbf{x}) + \rho\phi(\mathbf{x}),$$

where a large ρ discourages constraint violations but may hinder exploration. This method is simple but may require careful tuning of ρ to balance feasibility and convergence. The adaptive penalty method dynamically adjusts the penalty coefficient based on the iteration count or constraint violations. A commonly used adaptive penalty function is:

$$F(\mathbf{x}, t) = f(\mathbf{x}) + (C(t + 1))^\alpha \phi(\mathbf{x}),$$

where C is a scaling factor, t is the iteration number, and α controls the rate of penalty increase. This method allows the optimizer to explore the search space more freely in early iterations and gradually enforce feasibility as the optimization progresses.

2.2 Parameter Tuning Studies

The bump test function (**P3**) was used to tune the parameters of the Particle Swarm Optimization (PSO) algorithm. The function is defined as:

$$\begin{aligned} \text{Minimize:} \quad & - \left| \sum_{i=1}^n \cos^4(x_i) - 2 \prod_{i=1}^n \cos^2(x_i) \right| / \sqrt{\sum_{i=1}^n i x_i^2}, \\ \text{Subject to:} \quad & \prod_{i=1}^n x_i > 0.75, \quad \sum_{i=1}^n x_i < 15n/2, \quad 0 \leq x_i \leq 10. \end{aligned}$$

The goal of this section is to optimize the PSO hyperparameters for this function. The parameters tuned include:

- Number of particles (n_p)
- Inertia weight (w)
- Cognitive coefficient (c_1)
- Social coefficient (c_2)
- Static penalty parameter (ρ) for the static penalty method
- Adaptive penalty parameters (C and α) for the adaptive penalty method

Hyperparameter Optimization Procedure

The performance of PSO with the specified hyperparameters was evaluated over 10 independent runs. The search space for the hyperparameters was defined as follows:

- Penalty method: {Static, Adaptive}
- n_p : Integer in $[10, 60]$
- w : Real in $[0.4, 1.4]$
- c_1 : Real in $[1.5, 2.0]$
- c_2 : Real in $[2.0, 2.5]$
- ρ : Integer in $[10^2, 10^4]$ (Static penalty only)
- C : Real in $[0.5, 1.0]$ (Adaptive penalty only)
- α : Real in $[1.0, 2.0]$ (Adaptive penalty only)

The `gp_minimize` function from the `scikit-optimize` library was used to perform Bayesian optimization with Gaussian process regression. The hyperparameter search involved:

- 200 total evaluations (`n_calls=200`)
- 20 random initial configurations (`n_random_starts=20`)
- Parallelized evaluations (`n_jobs=-1`)

The best hyperparameters obtained for each penalty method are summarized in the following tables:

Table 1: Top 3 Hyperparameter Configurations for STATIC Penalty Method

n_p	w	c_1	c_2	ρ	Best solution	Function value
59	0.406	1.527	2.135	7535	[1.60088, 0.4685]	-0.36498
59	0.499	1.527	2.014	2855	[1.60076, 0.46854]	-0.36497
13	0.412	1.985	2.071	9414	[1.60133, 0.46861]	-0.36477

Table 2: Top 3 Hyperparameter Configurations for ADAPTIVE Penalty Method

n_p	w	c_1	c_2	C	α	Best solution	Function value
42	0.625	1.968	2.001	0.50	1.94	[1.59319, 0.48156]	-0.35564
60	0.726	1.988	2.003	0.57	1.98	[1.68511, 0.46405]	-0.34221
59	1.355	1.974	2.481	0.98	1.44	[1.50011, 0.53947]	-0.31763

Convergence Trends

The convergence plots for the best configurations of each penalty method are shown below. These plots illustrate the mean convergence trend over 10 independent runs, with a confidence interval of ± 3 standard deviations.

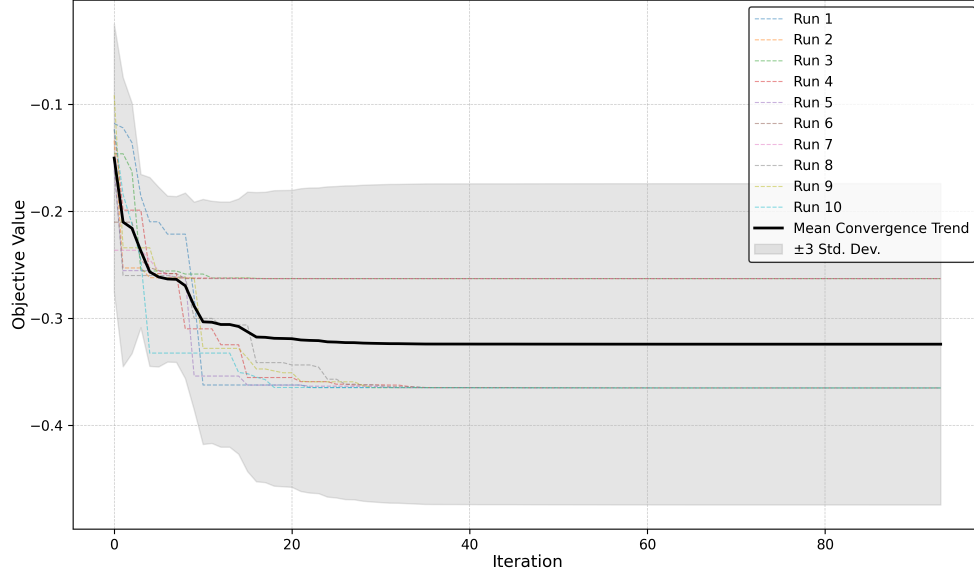


Figure 1: Convergence plot for the best STATIC penalty method configuration

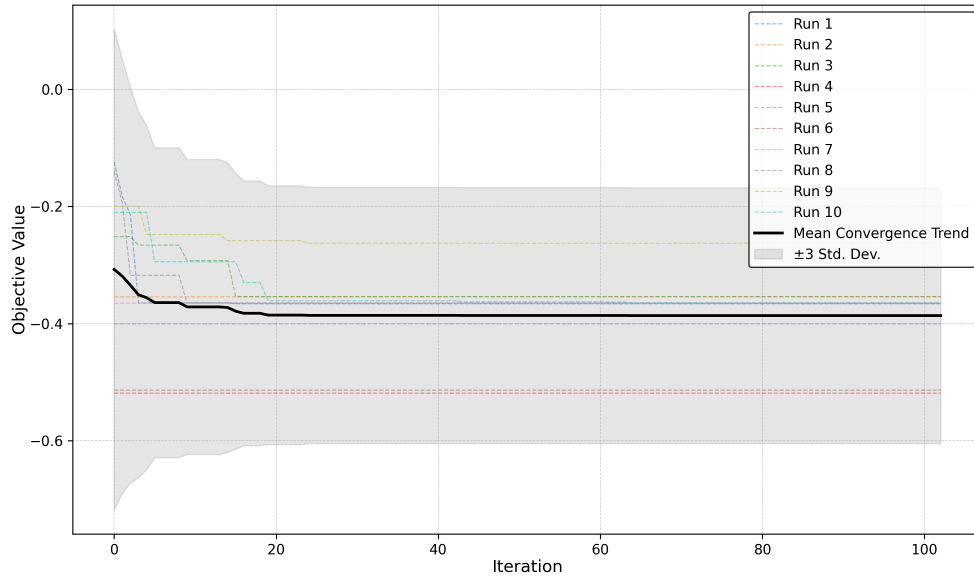


Figure 2: Convergence plot for the best ADAPTIVE penalty method configuration

Summary

From the results, the static penalty method achieved slightly better convergence performance for the bump test function (**P3**), with a best function value of -0.36498 . However, the adaptive penalty method showed comparable performance with a best function value of -0.35564 . The following tuned hyperparameters will be used in subsequent case studies to assess performance on other constrained optimization problems:

- Penalty method: Static
- n_p : 59
- w : 0.406
- c_1 : 1.527
- c_2 : 2.135
- ρ : 7535

2.3 Results for Case Studies

The tuned PSO parameters were applied to additional case studies from the "Case Studies" document. Results include the best solution found, the corresponding objective function value, and the feasibility of the solution. Detailed convergence plots and solution statistics are included for each case study.

2.3.1 P1: Minimization of the Rosenbrock Function

The Rosenbrock test function was minimized for both 2-dimensional and 5-dimensional cases. The problem is defined as:

$$\begin{aligned} \text{Minimize: } f(\mathbf{x}) &= \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2], \\ \text{Subject to: } -5 &\leq x_i \leq 5, \quad i = 1, 2, \dots, n. \end{aligned}$$

2-Dimensional Case The PSO algorithm successfully minimized the Rosenbrock function in 2 dimensions, achieving the following results:

- **Best solution:** $[0.9999, 0.9997]$
- **Best objective value:** 2.67×10^{-8}

5-Dimensional Case For the 5-dimensional case, the PSO algorithm required an increased number of iterations (maximum iterations set to 1000) to achieve convergence. The results are as follows:

- **Best solution:** $[0.9952, 0.9907, 0.9819, 0.9640, 0.9294]$
- **Best objective value:** 0.00176

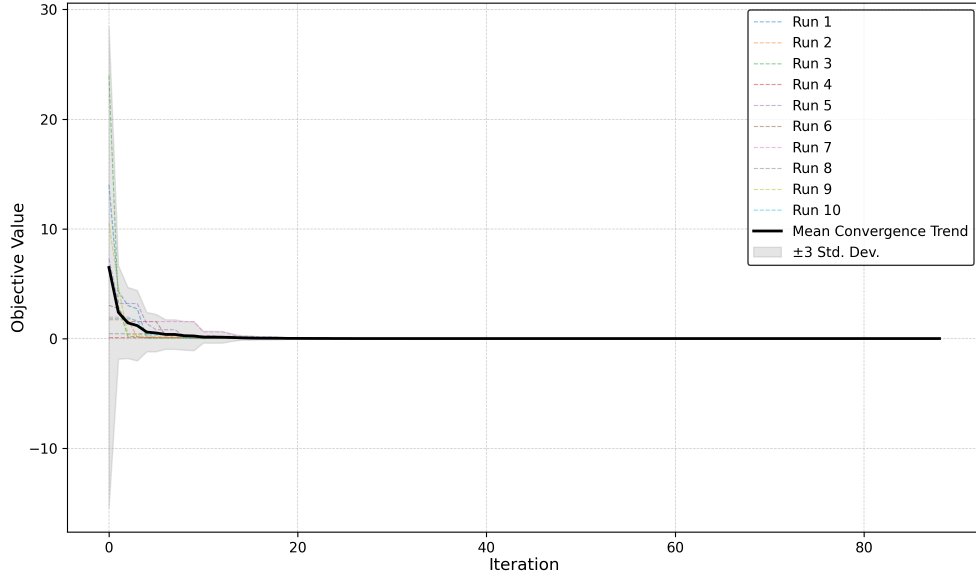


Figure 3: Convergence plot for P1 2D (2-dimensional Rosenbrock function)

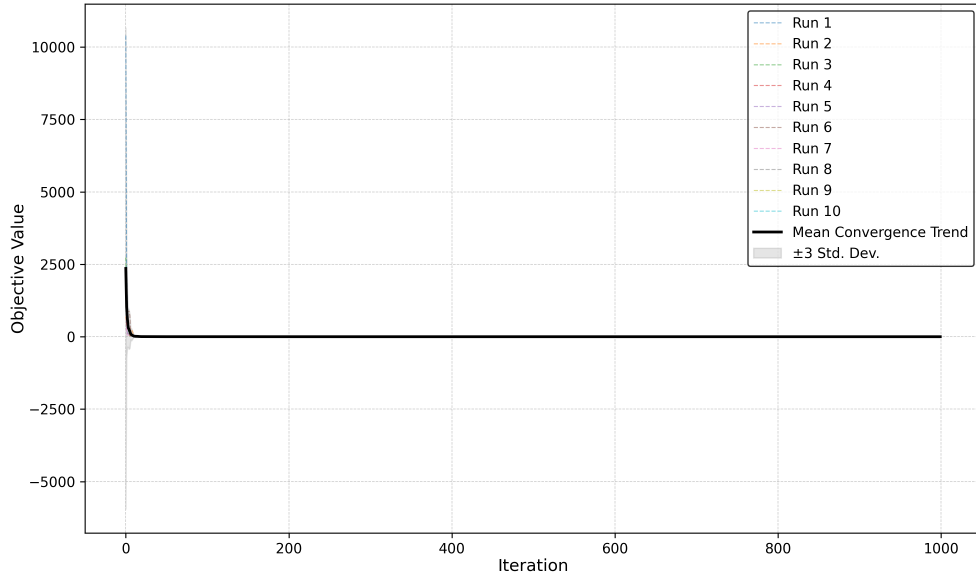


Figure 4: Convergence plot for P1 5D (5-dimensional Rosenbrock function)

Commentary on Results The results demonstrate that the tuned PSO parameters perform well in optimizing the Rosenbrock function. For the 2-dimensional case, the objective value achieved was close to zero, indicating near-perfect convergence to the global minimum. In the 5-dimensional case, the algorithm converged to a slightly higher objective value, which is expected due to the increased dimensionality and complexity of the problem. The convergence trends in both cases show smooth progress toward the optimal solution, with no major oscillations or divergence.

2.3.2 P2: Constrained Optimization Problem (Algebraic in 2 Variables)

The constrained optimization problem P2 was solved using the tuned PSO parameters. The problem is defined as:

$$\begin{aligned} \text{Minimize: } & f(\mathbf{x}) = x_1^2 + 0.5x_1 + 3x_1x_2 + 5x_2^2, \\ \text{Subject to: } & 3x_1 + 2x_2 + 2 \leq 0, \\ & 15x_1 - 3x_2 - 1 \leq 0, \\ & -1 \leq x_1 \leq 1, \\ & -1 \leq x_2 \leq 1. \end{aligned}$$

The PSO algorithm successfully converged to a solution within the specified tolerance. The results are as follows:

- **Best solution:** $[-0.8059, 0.2089]$
- **Best objective value:** -0.04032
- **Constraints:** $3x_1 + 2x_2 + 2 \approx -7.87 \times 10^{-5}$ and $5x_1 - 3x_2 - 1 \approx -13.73$

The convergence plot for this problem is shown below:

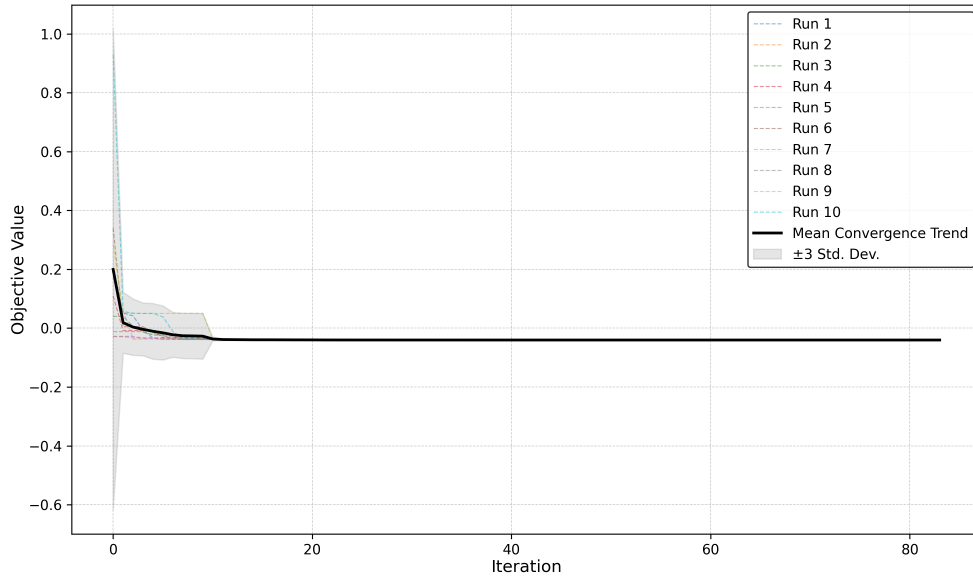


Figure 5: Convergence plot for P2 (Constrained optimization problem in 2 variables)

Commentary on Results The PSO algorithm effectively handled the constraints of the problem using the static penalty method. The best solution found is feasible, satisfying all constraints. The convergence plot demonstrates steady improvement toward the optimal solution, achieving convergence within the tolerance threshold. The negative objective value indicates the minimization of the function was successful under the given constraints.

2.4 Results for P3: Constrained Bump Test Function

The analysis for the 2-dimensional case of the bump test function (**P3**) was provided in the parameter tuning studies section. This section presents the results for the 10-dimensional case ($n = 10$) of **P3**.

The optimization was performed with a maximum of 300 iterations and a patience of 25 consecutive iterations to reach the tolerance threshold. The results for the 10-dimensional case are as follows:

- **Best solution:** $[3.1217, 3.0709, 3.0136, 2.9659, 1.4663, 0.3645, 0.3612, 0.3601, 0.3621, 0.3477]$
- **Best objective value:** -0.7473
- **Constraint 1 value:** $0.75 - \prod_{i=1}^n x_i = -0.0001$
- **Constraint 2 value:** $\sum_{i=1}^n x_i - 15n/2 = -59.5660$

The convergence trend for this optimization is shown in Figure 6. The plot demonstrates the mean convergence trend over 10 independent runs, with a confidence interval of ± 3 standard deviations.

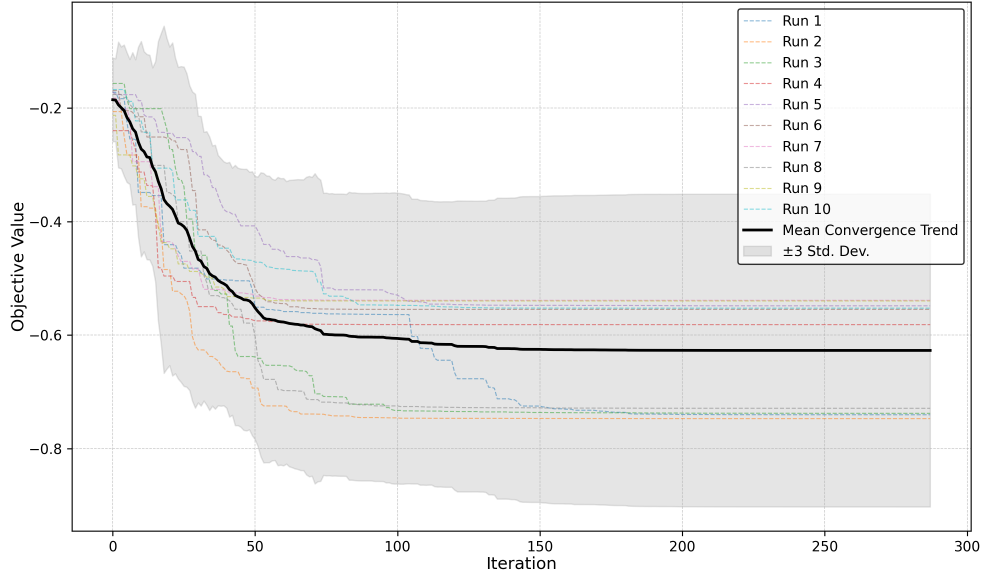


Figure 6: Convergence plot for **P3** with $n = 10$ using the tuned PSO parameters.

2.5 P4: Brachistochrone Problem

The Brachistochrone problem is a classic test case in optimization and was originally solved analytically by Johann Bernoulli. The objective is to determine the path between two points such that the travel time of a particle under the influence of gravity is minimized. For this study, the starting and ending points are specified as $(x_1, y_1) = (0, 1)$ and $(x_n, y_n) = (1, 0)$, respectively, and the coefficient of kinetic friction is set to $\mu_k = 0$.

2.5.1 Problem Formulation

To discretize the problem, the path is divided into n line segments, and the coordinates of the intermediate points are parametrized. The objective function for the total travel time is expressed as:

$$T = \sum_{i=1}^{n-1} \Delta t_i,$$

$$\Delta t_i = \frac{\sqrt{\Delta x_i^2 + \Delta y_i^2}}{\sqrt{h - y_{i+1} - \mu_k x_{i+1}} + \sqrt{h - y_i - \mu_k x_i}},$$

where:

- $\Delta x_i = \frac{x_n - x_1}{n-1} = \frac{1}{n-1}$ is the horizontal distance between consecutive points,
- $\Delta y_i = y_{i+1} - y_i$ is the vertical distance between consecutive points,
- $h = 1$ is the height of the starting point.

The goal is to minimize T by varying the y -coordinates of the intermediate points, y_i ($i = 2, \dots, n-1$), while keeping the first and last points fixed. This results in $n-2$ design variables, which we represent compactly with the vector $\mathbf{y} = [y_2 \ \dots \ y_{n-1}]^T$. The optimization problem can be written as:

$$\begin{aligned} \text{Find: } \arg \min_{\mathbf{y} \in \mathbb{R}^{n-2}} T(\mathbf{y}) &= \arg \min_{\mathbf{y} \in \mathbb{R}^{n-2}} \sum_{i=1}^{n-1} \frac{\sqrt{\Delta x_i^2 + \Delta y_i^2}}{\sqrt{h - y_{i+1}} + \sqrt{h - y_i}}, \\ \text{Subject to: } 0 &\leq y_i \leq 1, \quad i = 2, \dots, n-1. \end{aligned}$$

For validation, the analytical solution for the frictionless case ($\mu_k = 0$) is given by:

$$\begin{aligned} x &= a(\theta - \sin \theta), \\ y &= -a(1 - \cos \theta) + 1, \end{aligned}$$

where $a = 0.572917$ and $\theta \in [0, 2.412]$.

2.5.2 Results of PSO for the Brachistochrone Problem

The Particle Swarm Optimization (PSO) algorithm was used to solve the brachistochrone problem for two different levels of discretization: $n = 15$ and $n = 30$. The objective was to minimize the total travel time, T , by optimizing the vertical coordinates of the intermediate points while keeping the endpoints fixed.

Results for $n = 15$ The best solution obtained for $n = 15$ is:

$$\mathbf{y} = \begin{bmatrix} 0.7499, 0.6334, 0.5393, 0.4593, 0.3891, 0.3267, 0.2705, \\ 0.2197, 0.1736, 0.1318, 0.0938, 0.0594, 0.0282 \end{bmatrix}.$$

The total travel time for the best solution with 15 points is:

$$T = 1.2966.$$

The convergence trend across 10 independent runs is shown in Figure 7, and the comparison between the numerical and analytical solutions is shown in Figure 8. The maximum difference in height y_i between the numerical solution and the analytical solution is 0.025865.

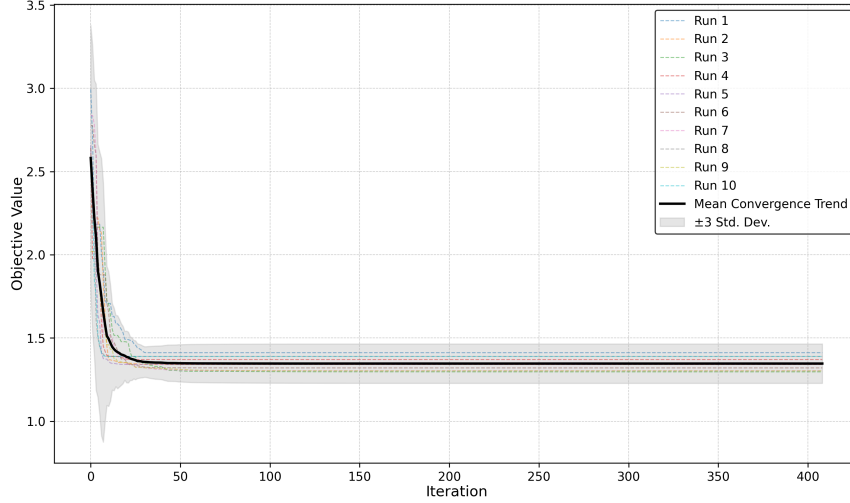


Figure 7: Convergence plot for the brachistochrone problem with $n = 15$.

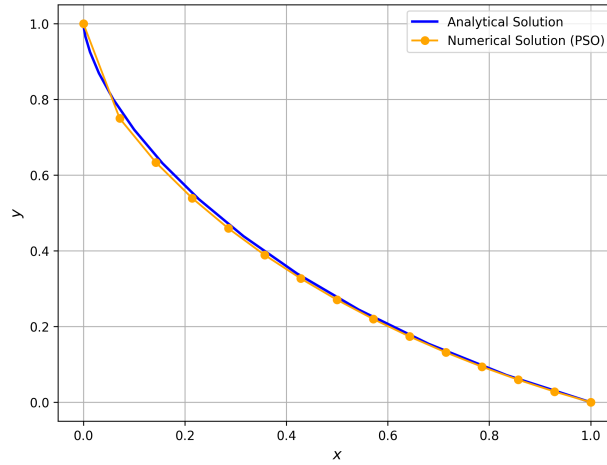


Figure 8: Comparison of numerical and analytical solutions for the brachistochrone problem with $n = 15$.

Results for $n = 30$ The best solution obtained for $n = 30$ is:

$$\mathbf{y} = \begin{bmatrix} 0.8398, 0.7631, 0.6988, 0.6446, 0.5947, 0.5499, 0.5094, 0.4717, 0.4370, 0.4049, \\ 0.3752, 0.3479, 0.3211, 0.2963, 0.2728, 0.2499, 0.2272, 0.2056, 0.1845, \\ 0.1641, 0.1447, 0.1258, 0.1067, 0.0881, 0.0690, 0.0511, 0.0336, 0.0167 \end{bmatrix}.$$

The total travel time for the best solution with 30 points is:

$$T = 1.2941.$$

The convergence trend across 10 independent runs is shown in Figure 9, and the comparison between the numerical and analytical solutions is shown in Figure 10. The maximum difference in height y_i between the numerical solution and the analytical solution is 0.019140.

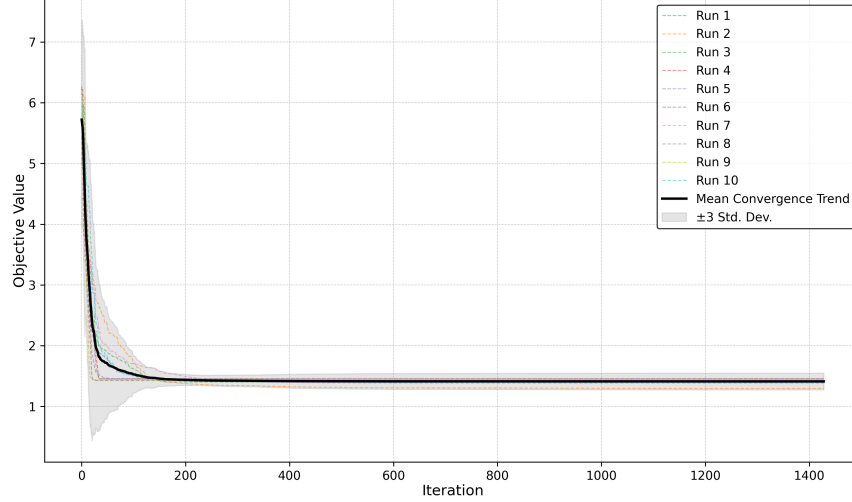


Figure 9: Convergence plot for the brachistochrone problem with $n = 30$.

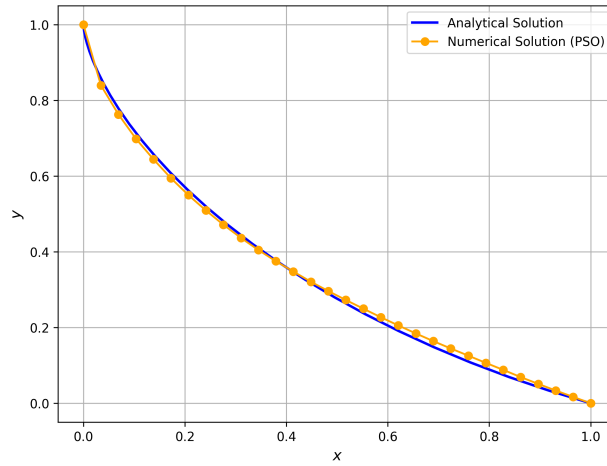


Figure 10: Comparison of numerical and analytical solutions for the brachistochrone problem with $n = 30$.

Summary The PSO algorithm successfully solved the brachistochrone problem for both levels of discretization. The numerical solutions closely matched the analytical solutions, with small differences observed due to the discretization and optimization process. The results validate the effectiveness of PSO for this problem.

2.6 P5: Estimation of Damping Factor and Stiffness

2.6.1 Problem Formulation

The objective of this study is to calibrate the parameters of a viscously damped linear oscillator to match the measured response u_{meas} of the system under periodic forcing. The analytical response of the oscillator is given by:

$$u(t) = (A \cos(\omega_d t) + B \sin(\omega_d t)) e^{-\frac{c}{2m}t} + \frac{F_0}{C} \cos(\omega t - \alpha),$$

where:

- $m = 1$ is the mass,
- k is the stiffness (unknown),
- c is the damping coefficient (unknown),
- $\omega = 0.1$ is the forcing frequency,
- $F_0 = 1$ is the forcing amplitude,
- $\omega_d = \sqrt{\frac{k}{m} - \left(\frac{c}{2m}\right)^2}$ is the damped natural frequency,
- $C = \sqrt{(k - m\omega^2)^2 + (c\omega)^2}$, and
- $\alpha = \tan^{-1}\left(\frac{c\omega}{k - m\omega^2}\right)$ is the phase angle.

The optimization problem is formulated as follows:

$$\text{Find: } \arg \min_{\mathbf{x} \in \mathbb{R}^2} \sum_{i=1}^N (u_{\text{sim}}(t_i; \mathbf{x}) - u_{\text{meas}}(t_i))^2,$$

$$\text{Subject to: } k \in [0.1, 100], \quad c \in [0.01, 10], \quad \frac{k}{m} - \left(\frac{c}{2m}\right)^2 \geq 0.$$

where:

- $u_{\text{sim}}(t_i; \mathbf{x})$ is the simulated displacement using parameters $\mathbf{x} = [k, c]$,
- $u_{\text{meas}}(t_i)$ is the measured displacement at time t_i , and
- $N = 500$ is the number of time points.

Instability Constraint: The constraint $\frac{k}{m} - \left(\frac{c}{2m}\right)^2 \geq 0$ ensures that the damped natural frequency ω_d is well-defined (i.e., the argument inside the square root remains non-negative). If this constraint is violated, the analytical solution is not physically meaningful, leading to numerical instabilities such as NaN values. The constraint is therefore enforced in the optimization process to ensure valid system parameters.

Choice of Bounds: The bounds for the parameters were chosen based on their physical meaning and practical engineering ranges:

- k (stiffness): Set to $[0.1, 100]$, covering a wide range from soft springs to very stiff ones.
- c (damping coefficient): Set to $[0.01, 10]$, accounting for lightly damped to heavily damped systems.

These bounds ensure the optimization process covers realistic parameter values while avoiding non-physical or unstable solutions.

2.6.2 Optimization Results

The tuned Particle Swarm Optimization (PSO) algorithm was applied to estimate the stiffness k and damping coefficient c by minimizing the squared error between the simulated response $u_{\text{sim}}(t)$ and the measured response $u_{\text{meas}}(t)$.

The optimization successfully converged to an optimal solution satisfying the physical constraints. The best solution obtained is:

$$\begin{aligned} k^* &= 10.2034, \\ c^* &= 0.7997. \end{aligned}$$

The corresponding objective function value (sum of squared errors) is:

$$J^* = 0.000376.$$

The convergence plot for the PSO optimization process is shown in Figure 11, illustrating the reduction in objective function value over iterations. The comparison between the simulated response using the optimized parameters and the measured response is presented in Figure 12.

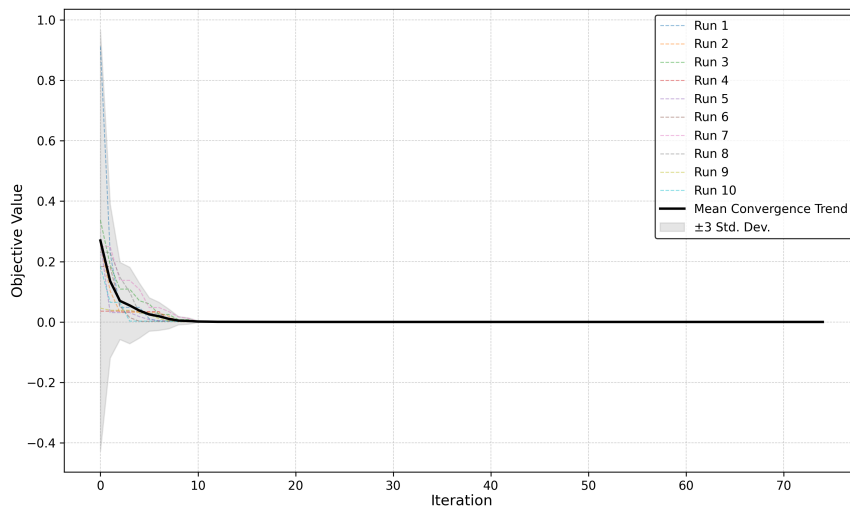


Figure 11: Convergence plot for the estimation of k and c using PSO.

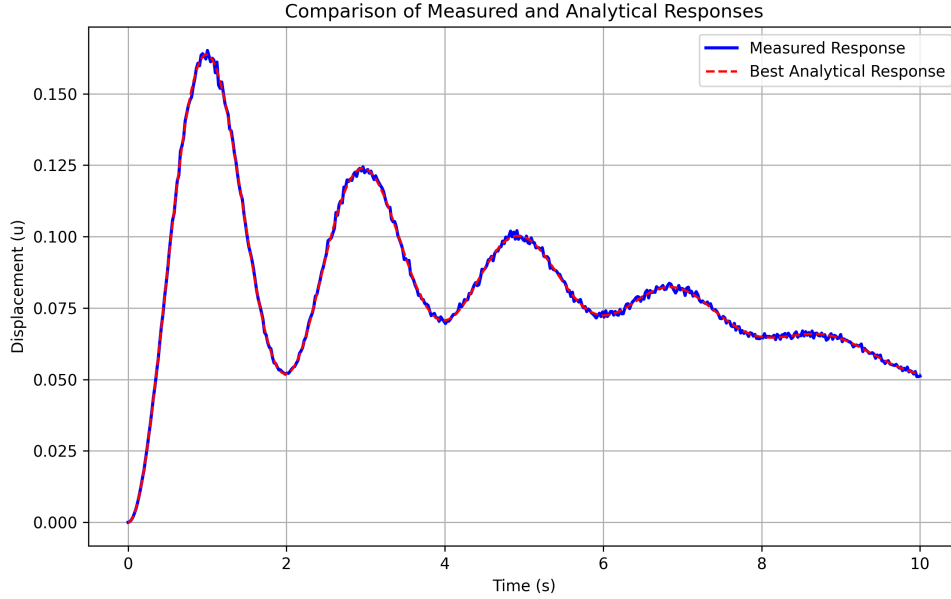


Figure 12: Comparison between the measured response and the simulated response using the optimized parameters.

3 Regression Problem

We start with the given regression model:

$$\hat{f}(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{i=1}^{M-1} w_i \phi_i(\mathbf{x}).$$

Given a dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, we seek to determine the optimal weights $\mathbf{w}$ by minimizing the following regularized weighted least-squares objective function:

$$J(\mathbf{w}) = \sum_{i=1}^N c_i \left(\hat{f}(\mathbf{x}^{(i)}, \mathbf{w}) - y^{(i)} \right)^2 + \sum_{i=1}^{M-1} \lambda_i w_i^2,$$

where $c_i > 0$ are user-defined weights and $\lambda_i > 0$ are regularization parameters.

3.1 Matrix Formulation

We introduce the following matrix definitions:

$$\Phi = \begin{bmatrix} 1 & \phi_1(\mathbf{x}^{(1)}) & \phi_2(\mathbf{x}^{(1)}) & \dots & \phi_{M-1}(\mathbf{x}^{(1)}) \\ 1 & \phi_1(\mathbf{x}^{(2)}) & \phi_2(\mathbf{x}^{(2)}) & \dots & \phi_{M-1}(\mathbf{x}^{(2)}) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \phi_1(\mathbf{x}^{(N)}) & \phi_2(\mathbf{x}^{(N)}) & \dots & \phi_{M-1}(\mathbf{x}^{(N)}) \end{bmatrix} \in \mathbb{R}^{N \times M}$$

$$\begin{aligned}
\mathbf{w} &= [w_0 \ w_1 \ w_2 \ \cdots \ w_{M-1}]^T \in \mathbb{R}^M \\
\mathbf{y} &= [y^{(1)} \ y^{(2)} \ \cdots \ y^{(N)}]^T \in \mathbb{R}^N \\
\mathbf{C} &= \text{diag}(c_1, c_2, \dots, c_N) \in \mathbb{R}^{N \times N} \\
\mathbf{\Lambda} &= \text{diag}(0, \lambda_1, \lambda_2, \dots, \lambda_{M-1}) \in \mathbb{R}^{M \times M}
\end{aligned}$$

3.2 Rewriting the Objective Function

In matrix notation, the objective function can be rewritten as:

$$J(\mathbf{w}) = \|\mathbf{C}^{1/2}(\mathbf{\Phi}\mathbf{w} - \mathbf{y})\|_2^2 + \mathbf{w}^T \mathbf{\Lambda} \mathbf{w}.$$

Expanding the first term:

$$J(\mathbf{w}) = (\mathbf{\Phi}\mathbf{w} - \mathbf{y})^T \mathbf{C} (\mathbf{\Phi}\mathbf{w} - \mathbf{y}) + \mathbf{w}^T \mathbf{\Lambda} \mathbf{w}.$$

3.3 Computing the Gradient

Expanding the squared term:

$$(\mathbf{\Phi}\mathbf{w} - \mathbf{y})^T \mathbf{C} (\mathbf{\Phi}\mathbf{w} - \mathbf{y}) = \mathbf{w}^T \mathbf{\Phi}^T \mathbf{C} \mathbf{\Phi} \mathbf{w} - 2\mathbf{y}^T \mathbf{C} \mathbf{\Phi} \mathbf{w} + \mathbf{y}^T \mathbf{C} \mathbf{y}.$$

Taking the gradient with respect to $\mathbf{w}$:

1. First term:

$$\nabla_{\mathbf{w}} (\mathbf{w}^T \mathbf{\Phi}^T \mathbf{C} \mathbf{\Phi} \mathbf{w}) = 2\mathbf{\Phi}^T \mathbf{C} \mathbf{\Phi} \mathbf{w}.$$

2. Second term:

$$\nabla_{\mathbf{w}} (-2\mathbf{y}^T \mathbf{C} \mathbf{\Phi} \mathbf{w}) = -2\mathbf{\Phi}^T \mathbf{C} \mathbf{y}.$$

3. Third term:

$$\nabla_{\mathbf{w}} (\mathbf{y}^T \mathbf{C} \mathbf{y}) = 0.$$

4. Regularization term:

$$\nabla_{\mathbf{w}} (\mathbf{w}^T \mathbf{\Lambda} \mathbf{w}) = 2\mathbf{\Lambda} \mathbf{w}.$$

Summing all terms:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = 2\mathbf{\Phi}^T \mathbf{C} \mathbf{\Phi} \mathbf{w} - 2\mathbf{\Phi}^T \mathbf{C} \mathbf{y} + 2\mathbf{\Lambda} \mathbf{w}.$$

Setting the gradient to zero for minimization:

$$2\mathbf{\Phi}^T \mathbf{C} \mathbf{\Phi} \mathbf{w} - 2\mathbf{\Phi}^T \mathbf{C} \mathbf{y} + 2\mathbf{\Lambda} \mathbf{w} = 0.$$

By rearranging the terms and simplifying, we get the desired system of equations:

$$(\mathbf{\Phi}^T \mathbf{C} \mathbf{\Phi} + \mathbf{\Lambda}) \mathbf{w} = \mathbf{\Phi}^T \mathbf{C} \mathbf{y}.$$

3.4 Solution for $\mathbf{w}$

To solve for the optimal weights $\mathbf{w}^*$, we can use Cholesky or QR factorization or singular value decomposition (SVD) depending on the rank of the left hand side matrix $\Phi^T \mathbf{C} \Phi + \Lambda$:

$$\mathbf{w}^* = (\Phi^T \mathbf{C} \Phi + \Lambda)^{-1} \Phi^T \mathbf{C} \mathbf{y}.$$

4 Constraint Relaxation Optimization Problem

Consider the constrained optimization problem:

$$\begin{aligned} & \min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \\ & \text{subject to: } g_j(\mathbf{x}) \leq 0, \quad j = 1, 2, \dots, m. \end{aligned}$$

It is given that the feasible set is empty. We introduce the vector $\boldsymbol{\delta} = (\delta_1, \delta_2, \dots, \delta_m)^T \in \mathbb{R}^m$ with relaxation parameters $\delta_j \geq 0$ to modify the constraints as follows:

$$g_j(\mathbf{x}) \leq \delta_j, \quad j = 1, 2, \dots, m.$$

The notation $\boldsymbol{\delta} \geq 0$ means that each component satisfies $\delta_j \geq 0$ for all j .

4.1 Minimizing the Total Relaxation

To find the smallest possible values of δ_j such that the feasible set is non-empty, we formulate the optimization problem:

$$\min_{\boldsymbol{\delta} \geq 0} \sum_{j=1}^m \delta_j$$

subject to the existence of $\mathbf{x} \in \mathbb{R}^n$ satisfying:

$$g_j(\mathbf{x}) \leq \delta_j, \quad j = 1, 2, \dots, m.$$

The objective function minimizes the sum of the relaxation parameters, which corresponds to the ℓ_1 -norm $\|\boldsymbol{\delta}\|_1 = \sum_{j=1}^m \delta_j$. The ℓ_1 -norm is used because it promotes sparsity in the relaxation parameters, meaning that only the most necessary constraints are relaxed. Additionally, it ensures a convex and computationally tractable optimization problem.

4.2 Minimizing the Number of Relaxed Constraints

To ensure feasibility while relaxing the minimum number of constraints, we introduce binary variables $z_j \in \{0, 1\}$, where $z_j = 1$ if constraint j is relaxed (i.e., $\delta_j > 0$). The problem is formulated as:

$$\begin{aligned} & \min_{z_j \in \{0, 1\}, \delta_j \geq 0} \sum_{j=1}^m z_j \\ & \text{subject to: } g_j(\mathbf{x}) \leq \delta_j z_j, \quad j = 1, 2, \dots, m. \end{aligned}$$

This formulation, however, does not differentiate between constraints that require significant relaxation and those requiring minimal adjustments. To address this, we refine the formulation by incorporating a penalty for larger relaxation values:

$$\min_{z_j \in \{0,1\}, \delta_j \geq 0} \sum_{j=1}^m \left(z_j + \lambda \frac{\delta_j}{\delta_{\max}} \right)$$

subject to: $g_j(\mathbf{x}) \leq \delta_j z_j, \quad j = 1, 2, \dots, m.$

where $\lambda > 0$ is a weighting parameter balancing the number of relaxed constraints and the magnitude of relaxation, and $\delta_{\max} = \max_j \delta_j$ ensures proper scaling.

This refined formulation ensures that constraints requiring only minor adjustments contribute less to the objective, thereby achieving a more balanced and practical relaxation strategy.

Solving these formulations poses computational challenges. The first formulation is a mixed-integer optimization problem, which is inherently combinatorial and difficult to solve efficiently for large-scale problems. The second formulation, though more balanced, introduces additional complexity due to the continuous relaxation penalty, requiring careful parameter tuning and scaling strategies to ensure numerical stability and tractability.

4.3 Minimizing the Cost of Relaxation

Given relaxation costs $c_j > 0$, the goal is to minimize the total cost of relaxation:

$$\min_{\delta \geq 0} \sum_{j=1}^m c_j \delta_j$$

subject to the existence of $\mathbf{x} \in \mathbb{R}^n$ satisfying:

$$g_j(\mathbf{x}) \leq \delta_j, \quad j = 1, 2, \dots, m.$$

5 Bonus: Particle Swarm Optimization on P3 ($n = 50$)

In this bonus section, we apply the Particle Swarm Optimization (PSO) algorithm to the constrained bump test function *P3* with $n = 50$. The best solution found, along with the corresponding optimal vector x^* , is saved in the attached ASCII file *best_solution_P3_n50.txt*. To verify the reported solution, one can evaluate the objective function and constraints at x^* using the provided script *verify_P3_n50.py*.